Network Policies

Securing Pod-to-Pod Communication — Module 8 of 13

Enterprise EKS Platform

📋 Module Agenda

- Kubernetes Networking Fundamentals
- Network Policy Basics & Anatomy
- Ingress and Egress Rules
- Common Policy Patterns
- CNI and Network Policy Providers
- Real-World Security Strategies
- Debugging & Monitoring Network Policies

Kubernetes Networking Fundamentals

Understanding the Flat Network Model

🌐 The Kubernetes Flat Network

Core Networking Requirement

Every pod gets its own IP address, and **every pod can reach every other pod** without NAT — across nodes, across namespaces.

- **Pod-to-Pod:** Direct communication via cluster-internal IPs
- **Cross-Namespace:** No isolation boundary by default
- **Cross-Node:** CNI plugin handles overlay or native routing
- **DNS Resolution:** Every service discoverable via CoreDNS

⚠️ The Security Problem: Default Allow

Default Behavior: Every pod can communicate with every other pod on every port. There are no firewalls between workloads.

🔴 Without Network Policies

- Compromised frontend reaches database directly
- Dev namespace can access production
- Lateral movement and data exfiltration are trivial

🟢 With Network Policies

- Frontend only reaches its API server
- Namespaces isolated; egress restricted
- Blast radius is contained

Network Policy Basics

Anatomy, Selectors, and Rules

Pod Selectors and Label Matching

matchLabels (AND logic)

Targets pods that match *all* listed labels — e.g. both `app` and `tier`.

matchExpressions (flexible)

Set-based selection with `In`, `NotIn`, `Exists`, `DoesNotExist` operators.

Empty podSelector `{}` — Selects *all* pods in the namespace. This is how you create default deny policies.

📥 Ingress Rules: Controlling Inbound Traffic

Inbound sources come in three forms, which can be combined:

- **podSelector** — pods in the same namespace
- **namespaceSelector** — pods in other namespaces
- **ipBlock** — CIDR ranges, with optional `except`

> **OR vs AND:** Separate list items under `from:` are OR'd. Multiple selectors *within* a single list item are AND'd.

📤 Egress Rules: Controlling Outbound Traffic

Egress mirrors ingress structure, controlling where pods may connect:

- **DNS first** — allow UDP/TCP port 53 or name resolution breaks
- **Internal services** — podSelector to databases and APIs
- **External endpoints** — ipBlock CIDR on the required port

> **Never forget DNS!** If you restrict egress, you must explicitly allow UDP/TCP port 53 or your pods cannot resolve service names.

Common Policy Patterns

Recommended Patterns

🚫 Default Deny Policies

Deny All Ingress

Empty `podSelector` with `policyTypes: [Ingress]` and no rules blocks all inbound traffic in the namespace.

Deny All Egress

Same shape with `policyTypes: [Egress]` and no rules blocks all outbound traffic.

Best Practice: Apply both default-deny policies to every namespace first, then layer on allow rules. Denying egress breaks DNS -- always pair with a DNS allow policy.

🚫 Companion: Allow DNS with Egress Deny

A default-deny-egress policy must be paired with an egress allow rule for DNS — an empty `podSelector` permitting UDP and TCP port 53 to any namespace.

> **Tip:** Some teams scope DNS to `kube-system` only, but allowing any namespace is safer for custom DNS setups.

🔁 Allow Specific Pod-to-Pod Communication

Three-Tier Application Pattern

Frontend → API Server → Database — each tier only talks to its neighbors.

- API ingress allows only `tier: frontend` pods on port 8080
- Database ingress allows only `tier: api` pods on port 5432
- Frontend cannot reach the database directly

Result: A compromised pod can only reach its immediate neighbor, sharply limiting lateral movement.

CNI & Network Policy Providers

Implementation Behind the API

🔌 CNI Provider Comparison

Feature	Calico	Cilium	AWS VPC CNI
K8s NetworkPolicy	✅	✅	✅ (v1.25+)
Global Policies	✅	✅	❌
L7 Filtering (HTTP)	✅ (Enterprise)	✅	❌
DNS-Based Rules	✅	✅	❌
Host-Level Policies	✅	✅	❌
Encryption (WireGuard)	✅	✅	❌
Flow Logging	✅	✅ (Hubble)	VPC Flow Logs

☁️ AWS VPC CNI & Network Policy on EKS

Since EKS 1.25, the VPC CNI plugin natively enforces NetworkPolicy using eBPF.

- **Enable:** Update `vpc-cni` add-on with `enableNetworkPolicy: true`
- **eBPF-based:** Kernel-level enforcement with minimal latency
- **Standard API:** Uses native `networking.k8s.io/v1` resources
- **VPC Flow Logs:** AWS-native network flow visibility
- **Limitation:** L3/L4 only — no extended CRDs like Calico/Cilium

Need L7 or global policies? Run Calico as a policy-only overlay alongside VPC CNI.

🐝 Cilium: L7-Aware Network Policies

The `CiliumNetworkPolicy` CRD extends standard rules with application-layer awareness — beyond allowing TCP 8080, it can permit only specific HTTP methods and paths (e.g. `GET /api/v1/.*`).

L7 Filtering: Cilium restricts by HTTP method, path, headers, and gRPC method — far beyond standard NetworkPolicy.

Real-World Security Strategies

From Policy to Practice

Zero-Trust Networking in Kubernetes

Zero-Trust Principle

Never trust, always verify. Every connection is denied unless explicitly permitted, regardless of source location.

1. Default Deny

- Apply to every namespace
- Both ingress and egress
- Automate via admission controller

2. Least Privilege

- Allow only required paths
- Restrict ports precisely
- Scope to exact labels

3. Verify & Audit

- Log denied connections
- Review policies in CI/CD
- Test with network probes

🧩 Microsegmentation Strategies

Namespace-Level Segmentation

- Isolate by environment (dev/staging/prod)
- Isolate by team or business unit
- Broad but effective first step
- Restrict ingress to same-namespace pods

Application-Level Segmentation

- Isolate by microservice tier
- Per-service egress restrictions
- Granular but higher maintenance
- Scope ingress to exact app labels

🔍 Debugging Network Policies

- **Forgotten DNS:** Egress policies that don't allow port 53 break all name resolution silently
- **OR vs AND confusion:** Two selectors in one `from:` item are AND'd; separate `from:` items are OR'd
- **CNI doesn't enforce:** Some CNI plugins (e.g., Flannel) ignore NetworkPolicy entirely
- **Namespace-scoped:** A policy in namespace A cannot affect pods in namespace B

📊 Monitoring & Auditing Network Flows

🐝 Cilium Hubble

- Real-time flow visibility
- Policy verdict logging
- Service dependency maps

🐅 Calico Flow Logs

- Allow/deny log entries
- Export to SIEM systems
- Policy recommendation engine

☁️ AWS VPC Flow Logs

- ENI-level traffic capture
- CloudWatch + Athena queries
- Pod IP correlation

Recommendation: Enable flow logging *before* enforcing policies. Write policies based on observed traffic, not assumptions.

🚀 Network Policy Lifecycle

Recommended Workflow

Treat network policies as code — version, review, test, and automate via CI/CD.

1. **Observe:** Enable flow logging and map actual traffic patterns
2. **Draft:** Write policies based on observed flows, store in Git
3. **Validate:** Lint with `kubeval` or `kubectl --dry-run=server`
4. **Test:** Apply in a staging namespace with connectivity tests
5. **Enforce:** Apply to production with monitoring enabled
6. **Audit:** Periodically review denied flows and policy coverage

📚 Module 8 Summary

📜 Policy Fundamentals

- Kubernetes flat network = default allow
- NetworkPolicy controls L3/L4 traffic
- podSelector targets, policyTypes declares
- Policies are additive (union of allow rules)
- Empty selector = all pods in namespace

🛡️ Operational Strategy

- Default deny first, allow rules second
- Always permit DNS in egress policies
- Verify CNI enforces NetworkPolicy
- Observe traffic before enforcing
- Treat policies as code in CI/CD

👨‍💻 Lab 8: Network Policies

Hands-On Exercise

- Deploy a multi-tier application (frontend, API, database)
- Verify default allow — all pods can reach all pods
- Apply default-deny ingress and egress policies
- Create targeted allow rules for the three-tier communication path
- Apply default-deny egress and allow DNS egress policies
- Debug a broken policy (wrong selector and port)
- Test cross-namespace isolation

> **Duration:** ~30-40 minutes

🎯 Key Takeaways

1. Default deny is non-negotiable. Apply default-deny ingress and egress to every namespace.

2. Always allow DNS. Every egress policy must include UDP/TCP port 53. This is the most common cause of policy-related outages.

3. Know your CNI. Standard NetworkPolicy is L3/L4 only. For L7 filtering, DNS-based rules, or global policies, use Calico or Cilium extensions.

4. Observe before enforcing. Enable flow logging and map real traffic before writing policies.

? Questions & Discussion

Module 8: Network Policies

Up Next: Module 9 — Observability — Logging, Metrics & Tracing